TÉCNICO UNIVERSIDADE DE LISBOA

# VPPaaS - THE VIRTUAL POWER PLANT-AS-A-SERVICE

2026 Enterprise Integration course project

## Table of Contents

# 1. Project context

Your EI project is located in the innovative concept of ***Virtual Power Plant-as-a-Service*** (VPPaaS). For short, a VPPaaS is a cloud-based distributed power plant that aggregates the capacities of heterogeneous Distributed Energy Resources (DER)—such as solar panels, home batteries, electric vehicles, and others —for the purposes of enhancing power generation, as well as trading or selling power on the electricity market.

In a classical approach each DER acts as "Energy Silos" and a grid cannot "see" or control these assets as a unified group. The lack of a standardized integration layer prevents the aggregation of these small energy packets into a volume significant enough to influence the grid. The challenge of your project is to create a system that enables a unified view of generation and storage capacity. The following definition explains the expected functions for an VPPaaS system.

"A VPP is a network that integrates dispersed energy resources, orchestrating them to operate cohesively as an extensive power generation facility. The primary goal of a VPP is to enhance grid stability, efficiency, and reliability while mitigating emissions and costs associated with conventional power generation. Generally, the architecture of a VPP contains three essential elements. Firstly, a diverse array of energy assets, ranging from solar panels and wind turbines to energy storage systems and energy-efficient buildings, form the foundation of the VPP. Energy Management Systems (EMS) play a pivotal role in efficiently overseeing various energy resources, including dispatchable power plants, intermittent generation units, storage facilities, and demand response systems. The second component is the communications network, facilitating data exchange and control signals among different VPP elements. For instance, EMS facilitates energy trading within the VPP through bidirectional communication and real-time status updates. The third component, the control system, handles data collection and analysis, power market prediction, resource modelling, aggregation, and transaction decisions. It oversees the real-time operation of distributed energy production and consumption. Cloud-based software is used for data analysis, optimization, and decision-making, targeting cost reduction, pollution minimization, and profit maximization."

*From: Md Romyull Islam, Long Vu, Nobel Dhar, Bobin Deng, and Kun Suo. 2024. Building a Resilient and Sustainable Grid: A Study of Challenges and Opportunities in AI for Smart Virtual Power Plants. In Proceedings of the 2024 ACM Southeast Conference (ACMSE '24). Association for Computing Machinery, New York, NY, USA, 95–103. https://doi.org/10.1145/3603287.3651202*

Therefore, your project's goal is to develop an **information system to support the operation of a VPPaaS provider**. The VPPaaS is designed to offer unique benefits, such as stabilizing the grid during peak demand and allowing individual "Prosumers" (producers-consumers) to monetize their hardware. Additionally, the VPPaaS serves as an avenue for inter-zone cooperation, balancing energy loads between different geographic grid zones.
Each prosumer is connected to a grid operator by an asset link through a digital contract. To optimize its operation, multiple asset links can be contracted by a prosumer. VPPaaS facilitates the establishment of this commercial relationship between prosumers and grid operators and offers added value services to optimize the overall operating conditions benefiting both actors.

# 2. Core Concepts Definition

Prosumer - is an entity (household or business) that consumes electricity but also possesses generation or storage capabilities. They register in VPPaaS to monetize their assets.

Utility Operator – represents the energy manager of a specific grid cell that can establish the asset links with prosumers. They register in VPPaaS to offer their infrastructure energy supply services.

Grid Cell - represents a specific geographic or logical cluster (*e.g.*, "Lisbon-Downtown", "Porto-Industrial"). Each zone has specific capacity constraints and is managed by a specific utility operator.

Asset Link - is the digital contract that associates a specific Prosumer's hardware (*e.g.*, a Battery) with a specific Utility Operator. This link authorizes the VPPaaS to monitor and control that asset.

Telemetry - the system relies on high-frequency "Telemetry Events." These are data packets sent by the Asset Link containing real-time status (State of Charge, Current Output, Voltage, Connection Status, or others depending in the hardware). VPPaaS relies in telemetry to provide analytics and uses telemetry to produce decisions affecting the operational conditions of the grid.

Flexibility Event - is a command or offer generated by the VPPaaS. For example, if the grid is stressed, the system emits an event offering a financial incentive for prosumers to discharge energy.

Grid Balancing Recommendation - this concept involves identifying opportunities to shift load between Grid Zones. If Zone A is facing a deficit and Zone B has a surplus, the system recommends specific actions to balance the two.

The following Sections 0, 4, 0 and 0 detail the business architecture, the application integration reference architecture, the data integration architecture and the technological integration architecture, respectively.

# 3. Business architecture

An use case diagram of the VPPaaS is presented in

Figure 1 emphasizing the actors that interact with VPPaaS and stating the use cases supported by VPPaaS.

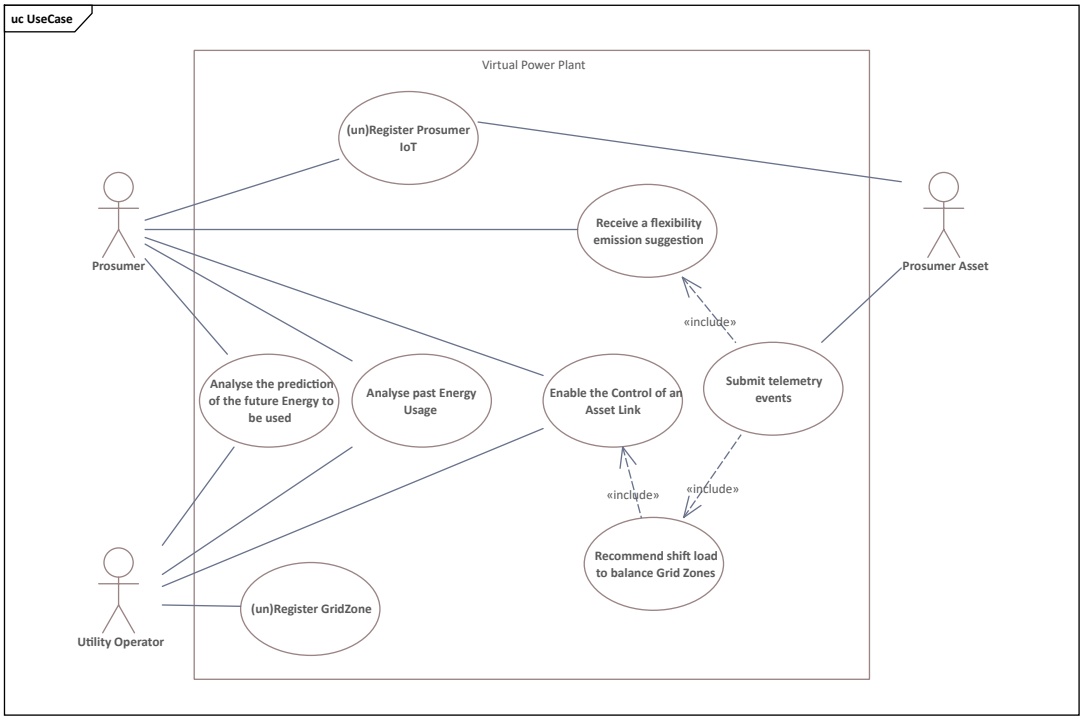


Figure 1. VPPaaS information system use case diagram represented in UML.

Two major business problems are targeted to be solved by your project.

a) The **onboarding** and **decomission** of prosumers, utility operators and prosumer assets. The viability of a VPP relies on the mass aggregation of thousands of small resources. The problem lies in the high friction of the Onboarding process. It is a multi-layered workflow that requires synchronizing Business Validation (KYC - Know Your Customer, contract signing, tariff selection) with Technical Provisioning (IoT device handshake, protocol compatibility checks, and capacity verification). Manual onboarding is unscalable; therefore, the system must automate the "handshake" between a new Prosumer's hardware (*e.g.*, a Tesla Powerwall) and the VPP control center, ensuring the asset is legitimate and controllable before it enters the market.
The Risk of "Zombie" Assets (Decommissioning): Equally critical is the Decommissioning process. In dynamic grid environments, prosumers move houses, sell electric vehicles, or switch energy providers. If an asset is physically disconnected but remains logically active in the VPP database, the system will attempt to dispatch commands to a "Phantom Asset." This leads to Grid Balancing Errors—where the VPP promises energy to the Utility Operator that it physically cannot deliver because the assets no longer exist. The challenge is to design a strict "Off-boarding" orchestration that instantly revokes security keys, stops telemetry ingestion, and updates the aggregate capacity calculations to maintain grid reliability.

b) The **emergency demand response** and **critical event handling.** The Latency and Concurrency Challenge: While standard grid balancing is driven by economic signals (hourly prices), Emergency Demand Response is driven by grid stability (frequency deviations or line overloads). In these scenarios, the Grid Operator (DSO/TSO) requires a load reduction of specific MegaWatts (MW) within seconds to prevent a blackout. The integration challenge here is Massive Concurrency. The VPP platform must receive a single "Critical Event" signal and instantly broadcast shut-down commands to thousands of disparate assets (EV chargers, Batteries, …) simultaneously.

# 4. Application integration reference architecture

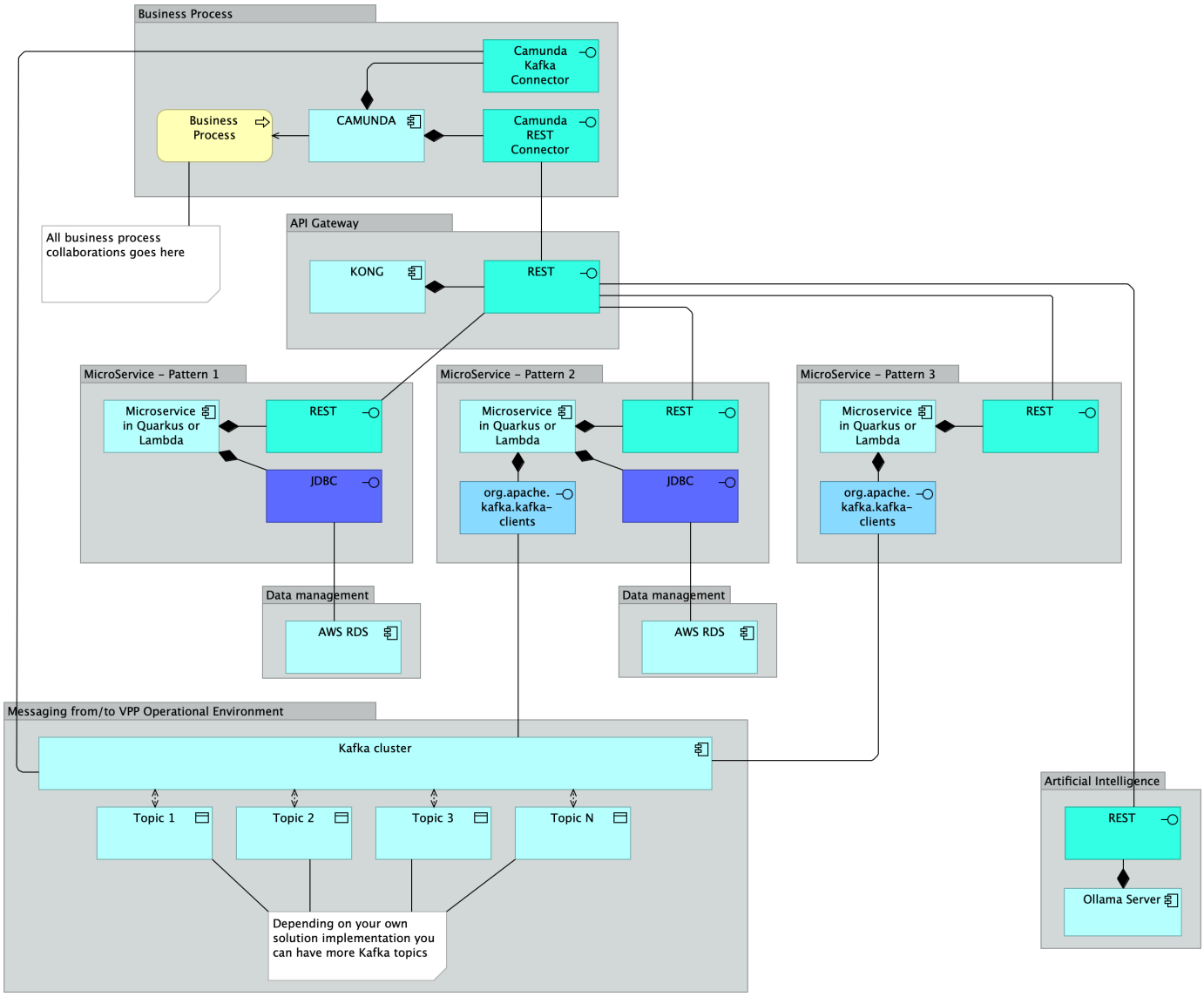


Figure 2. The expected application integration reference architecture patterns represented in ArchiMate. Each pattern is customized according to the needs of the requirements. From left to right. First pattern manages a business entity stored in a database using a JDBC interface and exposed by REST. Second, the most comprehensive, keeps a business data entity consistent between a Kafka cluster and a database, and exposes that composed entity through REST. Third pattern manages a business entity against a Kafka cluster by exposing a REST interface. Finally, artificial intelligence exposes an API-to-API gateway offering an Ollama server that is deployed at EC2 to support the business processes decisions.

# 5. Data integration architecture

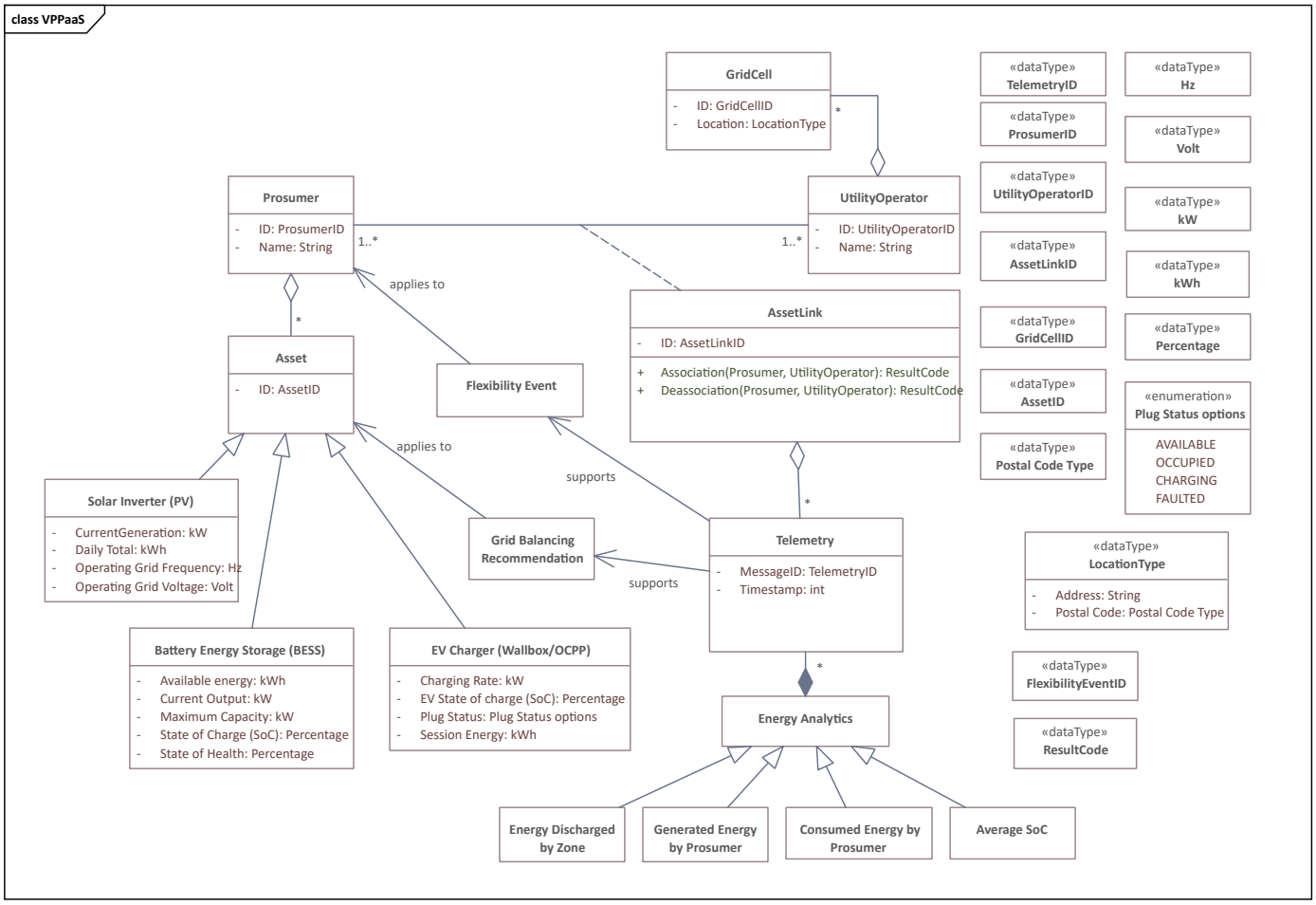


Figure 3. The UML domain model of VPPaaS system containing the most relevant classes and relationships. It should be considered as a baseline that is to be updated to each project' group need.

# 6. Technological integration architecture

This section instantiates the VPPaaS reference architecture (as presented in Figure 2) to the specific requirements that you are expected to develop. In the core of the IE course goals, and concomitantly, in the core of the project' reference architecture, locates the concept of a microservice. "*A microservice should be independently releasable services that are modelled around a business domain*". Therefore, the data integration architecture is key for understanding which microservices should be developed and how. In this line of reasoning, each one of classes depicted in Figure 3 represents one key concept taken from the VPPaaS domain that need to be managed by the VPPaaS system. From Figure 4 to Figure 11, the role of each microservice is depicted in the scope of the required application integrations and aligned with the classes in Figure 3.

## I. Prosumer Management

- Responsible for registering and managing Prosumers (Name, Fiscal ID, Location, Payment Details).
- Responsible to identify the assets of that prosumer

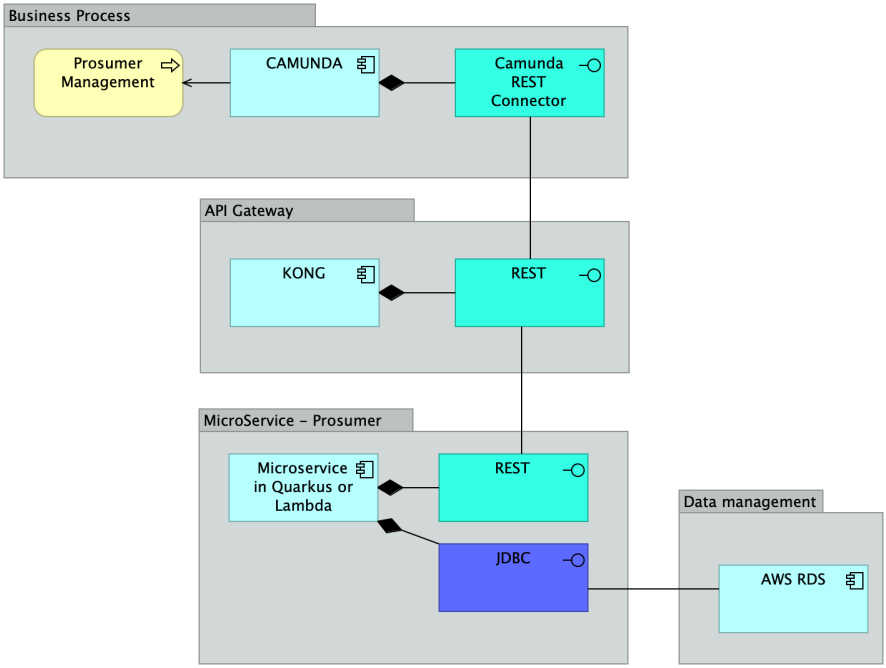


Figure 4. The integration architecture of Prosumer microservice.

## II. Utility Operator Management

- Responsible for registering and managing utility operators (Name, Fiscal ID, Location, Payment Details).
- Manages the lifecycle of Grid Zones (Creation, updating max capacity, setting geographic boundaries).
- Stores grid constraints (e.g., "Max Load: 50MW").

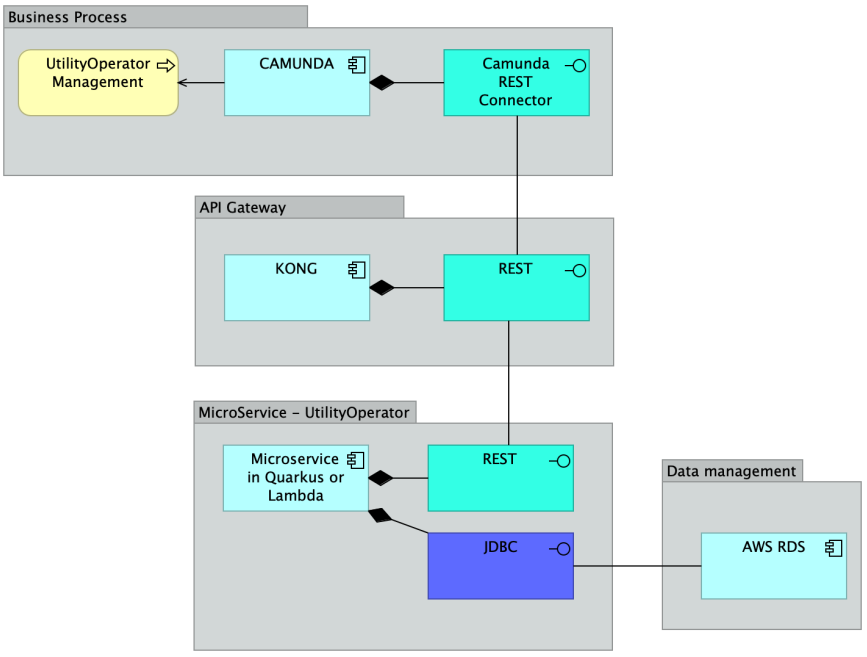


Figure 5. The integration architecture of Utility Operator microservice.

# III. Asset Link Management

- Manages the association between a Prosumer and a Grid Zone.

- Integration Requirement: When a link is active, it must publish to a Kafka Topic Asset-at-Zone (Format: AssetID-ZoneID) to enable telemetry ingestion for that specific pair.

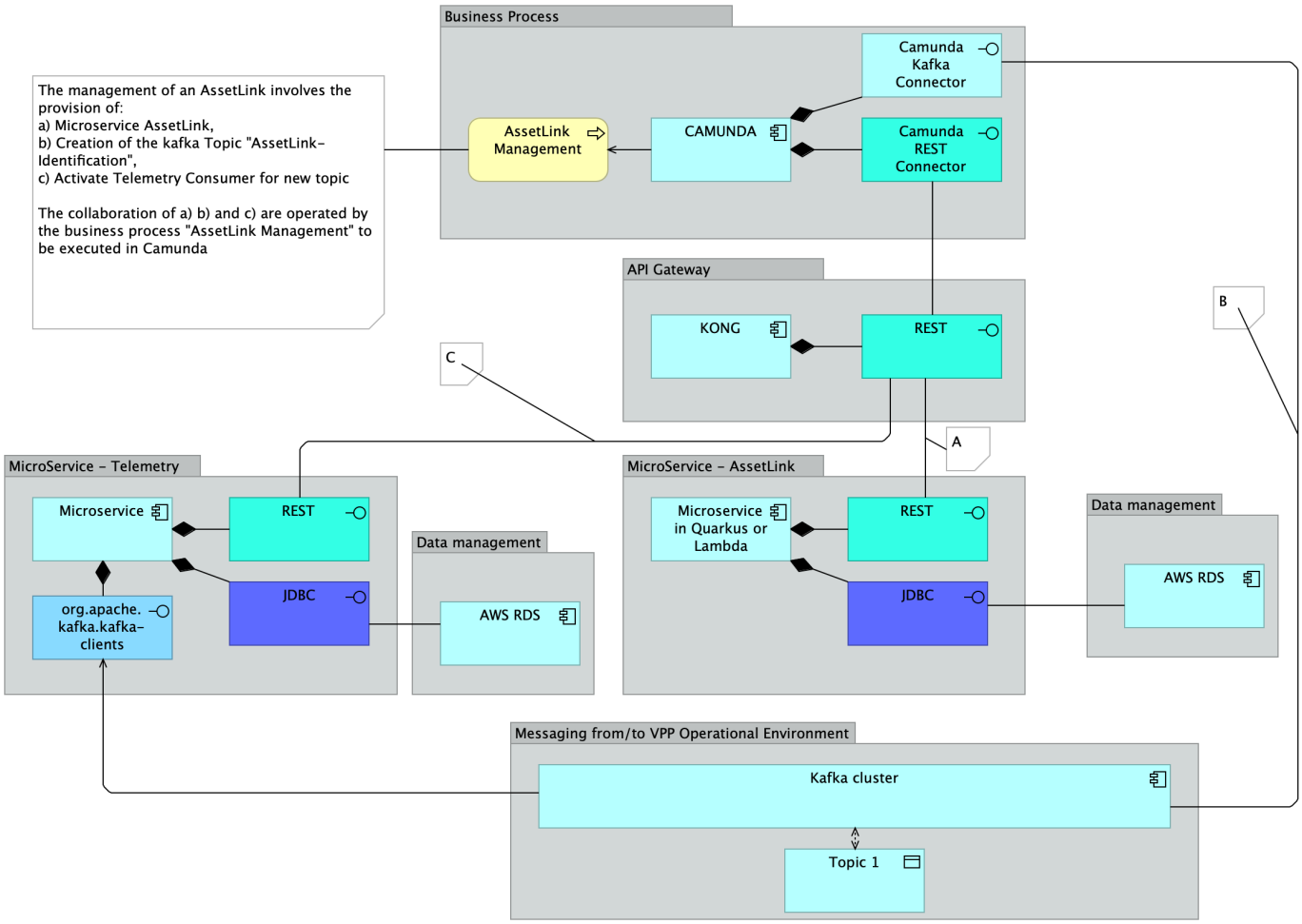


Figure 6. The integration architecture of AssetLink microservice.

## IV. Telemetry Ingestion

- High Volume Service: Consumes energy readings (kWh, voltage, status) from the prosumer's simulators. Data depends on the type Asset involved: Solar Inverter, Battery Energy Storage or EV charger
- Stores time-series data regarding asset performance.

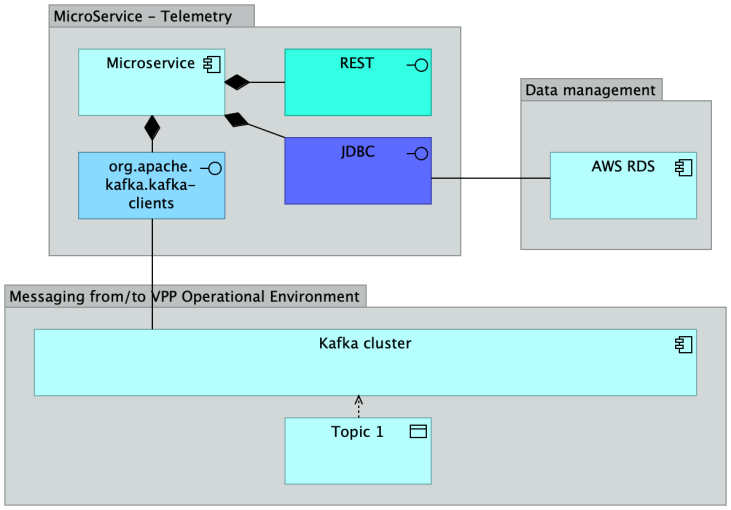


Figure 7. The integration architecture of Telemetry microservice.

## V. Flexibility Emission

- Analyses incoming telemetry. If an asset reports high capacity (>90% charge) during defined "Peak Hours," this service generates a "Flexibility Offer" and publishes it to the Flexibility-Offers Kafka topic.
- Example of a flexible emission rule:

```
Arbitrage Logic: If soc_percent > 90% AND current_market_price is HIGH -> Trigger Sell.
Balancing Logic: If soc_percent < 20% -> Mark as "Unavailable" for balancing requests.
```

OTHER:

```
Offer a financial incentive for discharge energy to a prosumer
```

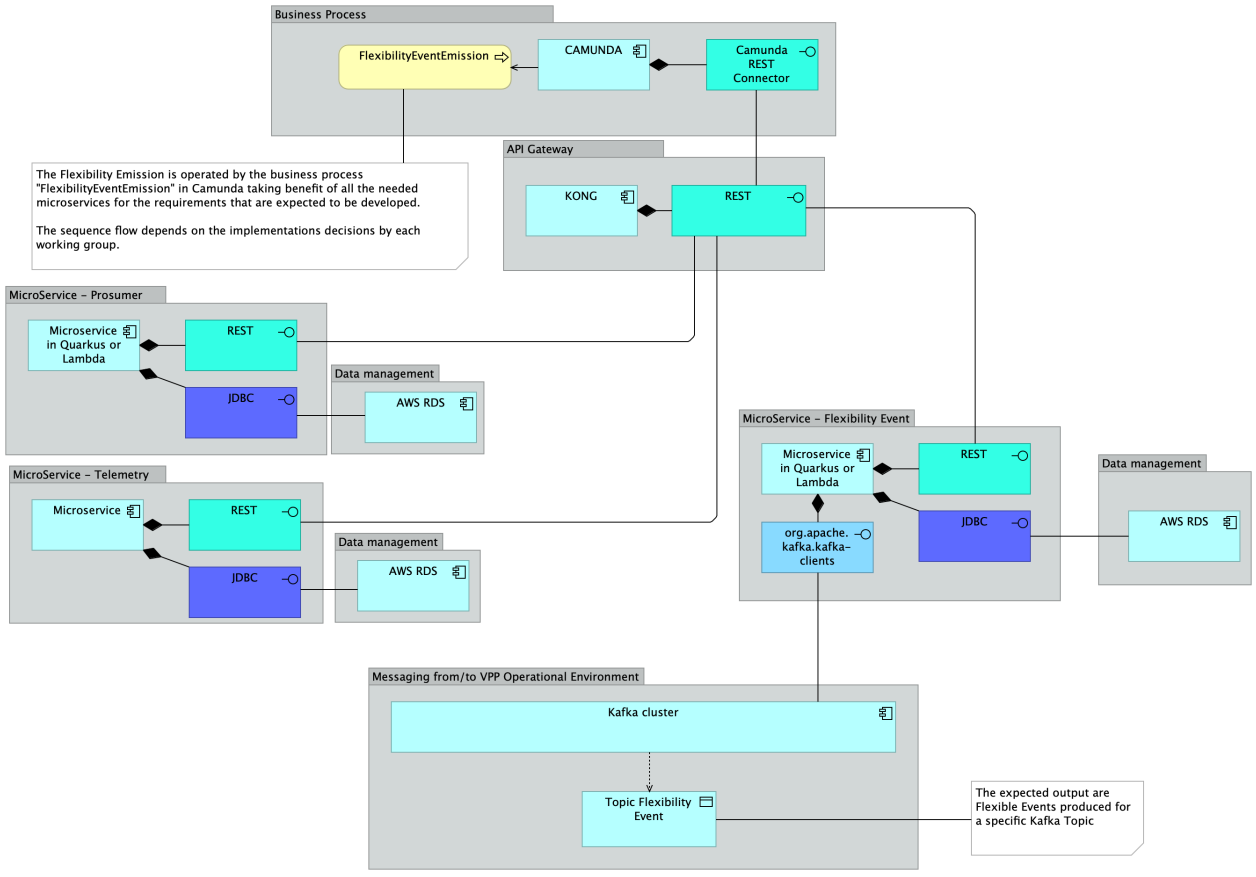


Figure 8. The integration architecture of Flexibility Event microservice.

# VI. Grid Balancing Recommendation

- Analyses aggregated data across different Grid Zones.

- If a Zone exceeds its safety threshold, this service scans neighbouring zones for surplus and produces a "Balancing Recommendation" event.

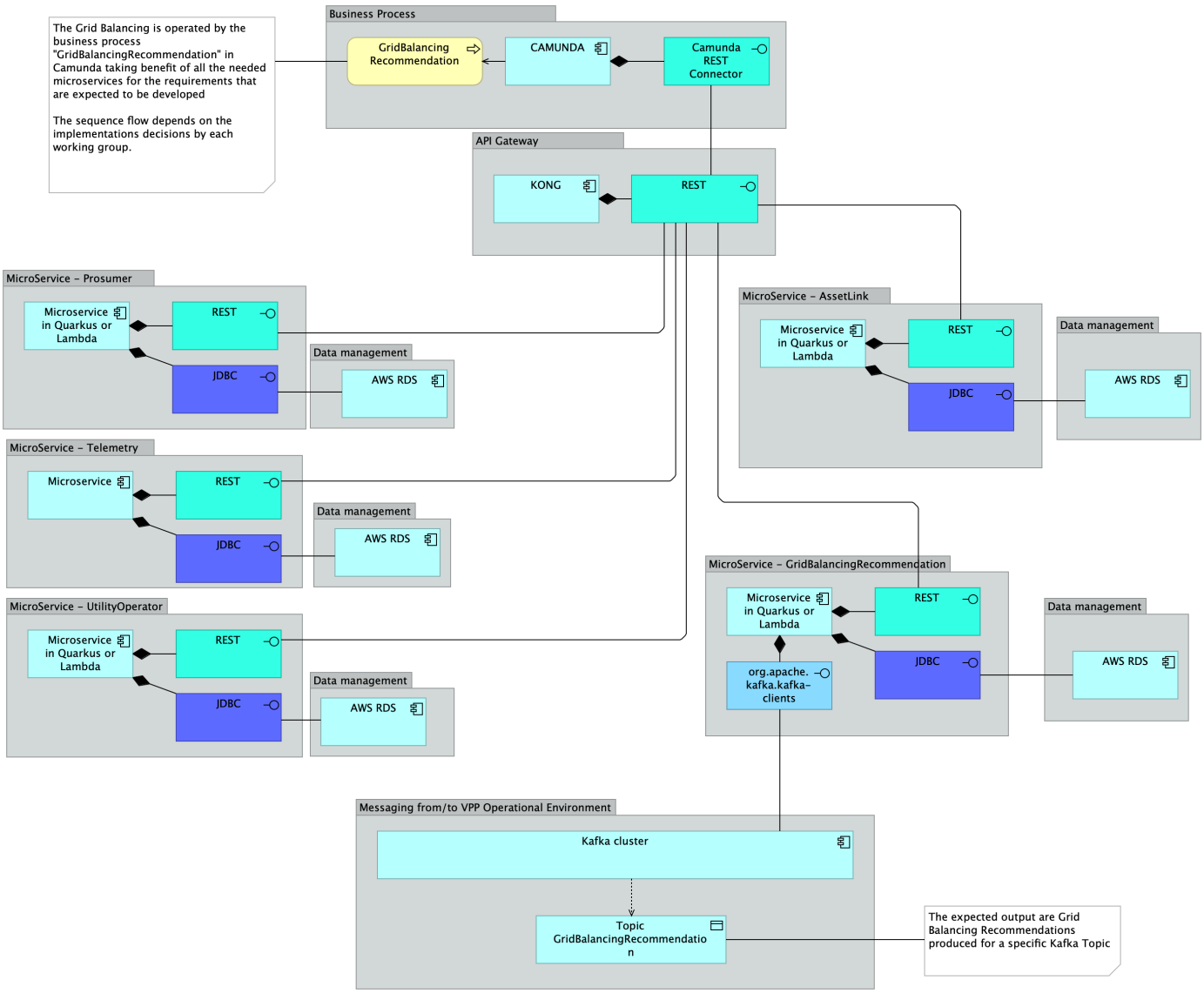


Figure 9. The integration architecture of Grid Balancing Recommendation microservice.

# VII. Energy Analytics

- Aggregates system-wide data: "Energy Discharged by Zone", "Generated Energy by Prosumer", "Consumed Energy by Prosumer","Average SoC" ….

- Produces calculated results to Kafka topics for dashboarding.

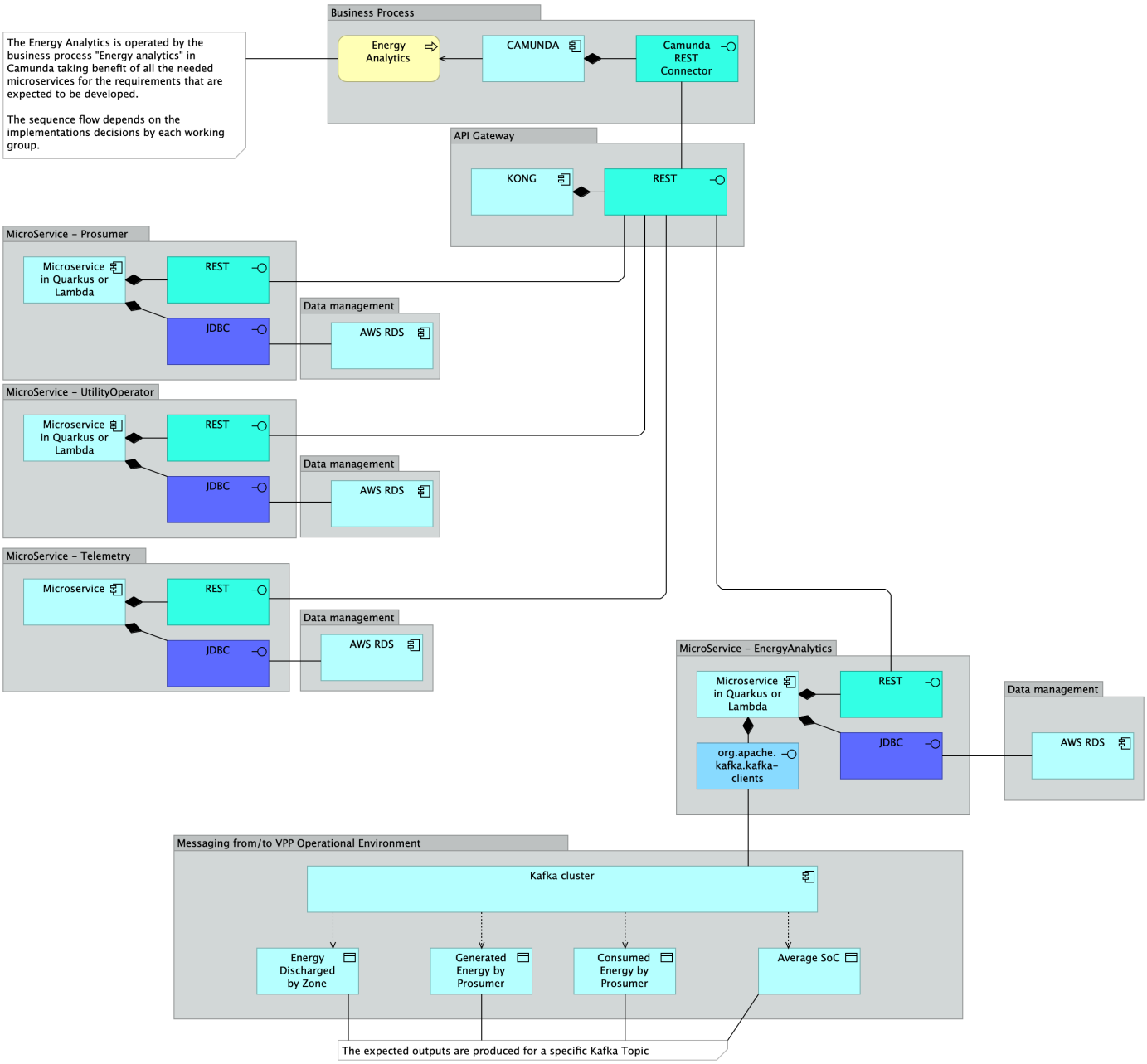


Figure 10. The integration architecture of Energy Analytics microservice.

# VIII. Flexibility Forecasting (AI)

The flexibility Forecasting encompasses:

- Use of a Large Language Model (Ollama) to analyze the unstructured logs of past Flexibility Events.

- Goal: Determine the sentiment and success rate of past events (e.g., analyzing log text to see if the "Discharge" command resulted in a successful "Grid Stable" state).

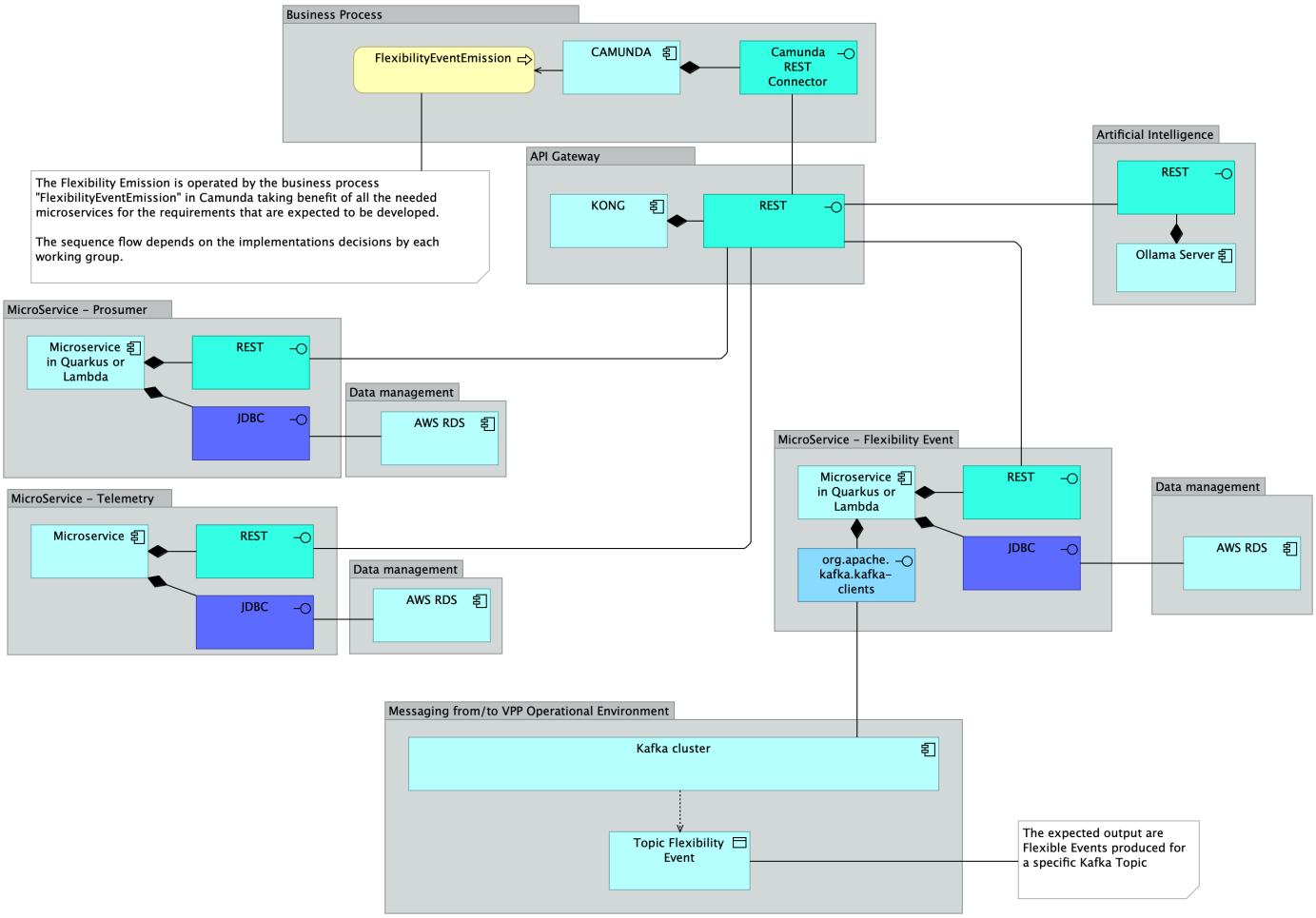


Figure 11. The integration architecture of Flexibility Event microservice extended to use AI.

# 7. Event producer tool

A VPP aggregates different devices (Solar, Batteries, EVs), the telemetry structure must be polymorphic or contain specific subsets of data. The event producer tool acts as a simulator of production and consumption using the asset links. The tool starts by discovering all the topic available in the kafka cluster (each topic is a different AssetLinkID-UtilityOperator) and then randomize messages for all that discovered topics.

The discovery assumes that each topic (*the pair: AssetLinkID-UtilityOperator*) has a name accordingly with the following format name:

```
AssetLinkID-UtilityOperator

For example: 560987123-EDP
```

Following is the detailed breakdown of telemetry readings, categorized by asset type, including the specific JSON fields.

The Common Header (Metadata) - Every telemetry packet must contain these fields to identify the source and context.

| Field | Type | Description |
|---|---|---|
| seqKey | UUID | Unique ID for the telemetry packet (for de-duplication). |
| timestamp | ISO8601 | Precise time of reading (e.g., 2025-05-17T10:00:00Z). |
| asset_id | String | The unique hardware ID (e.g., BATT-001). |
| asset_type | String | BATTERY, SOLAR or EV_CHARGER. |
| grid_cell_id | String | The ID of the zone this asset belongs to (e.g., LISBON-DT). |

Battery Energy Storage (BESS) - The most critical asset for your project. It enables the "Arbitrage" and "Balancing" use cases.

| Data Point | JSON Field | Unit | Why it's needed? |
|---|---|---|---|
| State of Charge | soc_percent | % | Core Logic. Determines if we can sell (High SoC) or need to buy (Low SoC). |
| Available Energy | energy_available_kwh | kWh | Calculates exactly how long we can discharge at full power. |
| Current Output | active_power_kw | kW | (+ve = Discharging, -ve = Charging). Used to verify if a command was obeyed. |
| Max Capacity | max_discharge_power_kw | kW | The "Speed limit" of the battery. We can't request 10kW from a 5kW battery. |
| State of Health | soh_percent | % | Long-term maintenance. If SoH < 70%, maybe exclude from aggressive trading. |
| Status | connection_status | Enum | One of the following: {ONLINE, OFFLINE, FAULT, MAINTENANCE} |

Solar Inverter (PV) - Used primarily for forecasting. We can't control the sun, but we can measure it.

| Data Point | JSON Field | Unit | Why it's needed? |
|---|---|---|---|
| Current Generation | generation_kw | kW | Real-time output. |
| Daily Total | daily_yield_kwh | kWh | For analytics/reporting. |
| Grid Voltage | ac_voltage_v | Volts | Critical for Grid Constraints. If Voltage > 253V, the inverter might trip (shut off). |
| Frequency | grid_frequency_hz | Hz | If Freq < 49.8Hz, the VPP might trigger an emergency discharge (Frequency Regulation). |

EV Charger (Wallbox/OCPP) - A "Controllable Load". We can't usually push power back (unless V2G), but we can stop charging.

| Data Point | JSON Field | Unit | Why it's needed? |
|---|---|---|---|
| Plug Status | connector_status | Enum | AVAILABLE, OCCUPIED, CHARGING, FAULTED. |
| Charging Rate | charging_power_kw | kW | The load we can "shed" (turn off) if the grid is stressed. |
| Session Energy | session_energy_kwh | kWh | For billing the user. |
| EV SoC | ev_soc_percent | % | Advanced: Only available if the car communicates it (ISO 15118). |

Examples of JSON produced message is given below.

### The BESS Payload (Standard):

topic = 560987123-EDP, with key=b0002dd9-3ab0-4a0b-9b4c-74790d7e3849


```
{
 "timeStamp":"2026-02-20 13:39:53.401",
 "asset_type":"BATTERY",
 "asset_id":"560987123",
 "grid_cell_id":"AUSTIN-DT",
 "payload": {
        "energy_available_kwh":13.936685931917559,
        "max_discharge_power_kw":2.947349764012512,
        "active_power_kw":-7.220102938275808,
        "connection_status":"MAINTENANCE",
        "soh_percent":31.943940694715046,
        "soc_percent":92.22976493508132
 }
}
```


### The Solar Payload (Grid Stress Indicator):

topic = 560987123-EDP, with key=f516b174-2c5f-405d-9891-450ed19c4e4c


```
{
 "timeStamp":"2026-02-20 15:00:30.638",
 "asset_type":"SOLAR",
 "asset_id":"560987123",
 "grid_cell_id":"CAIRO-ND",
 "payload": {
        "ac_voltage_v":254.69491311862646,
        "generation_kw":0.5640583520961961,
        "grid_frequency_hz":49.84521461297455,
        "daily_yield_kwh":63.7981840024461
 }
}
```

The EV Charger (Controllable Load):

`topic = 560987123-EDP, with key=440ce0b6-8ac7-4926-8650-36d43ca360fb`


```
{
 "timeStamp":"2026-02-20 15:20:49.816",
 "asset_type":"EV_CHARGER",
 "asset_id":"560987123",
 "grid_cell_id":"SANTOS-IN",
 "payload": {
        "charging_power_kw":0.9642079408177404,
        "connector_status":"AVAILABLE",
        "session_energy_kwh":13.627434098664882,
        "ev_soc_percent":50.49963512002662
 }
}
```


The tool is particularly useful to validate your solution. Start the tool accordingly with your Kafka configuration, and then, consume the produced messages from your architecture. To test performance, you can also increase the message throughput. **For further details, including the options available, please refer to** https://github.com/Enterprise-Integration-IST-2026/VPPaaS-EventProducer. **You are free to customize the simulator accordingly with your project' purposes.** If so, you can also create a pull request to change this repository.

**Moreover, Scenario 3 offers a baseline for the development of the core components of your project.**

**Refer to document of the VPPaaS Proof-of-Concept (PoC) -** *S3-Integration-Scenario.pdf* ***available at Fenix.***

# 8. Deliverables & Deadlines

## 1ST SPRINT

The first sprint consists in the definition of the information flows, in the setup of the kafka cluster, in the development of the microservices (that could be based on the already provided microservices) and testing of all the development components.

### The expected deliverables are:

1. Design your own understanding of the information flow considering the requirements and the architecture of VPPaaS. We suggest you specify the Kafka topics and partitions, as well as the microservices to be used using UML sequence diagrams (https://plantuml.com/) - but any other representation is accepted.

2. Setup and deployment of the Kafka Cluster using TERRAFORM
   Suggestion: for testing purposes use the VPPaaS producer (https://github.com/Enterprise-Integration-IST-2026/VPPaaS-EventProducer) and create the needed topics by command line

3. The implementation of the following microservices, using ((Quarkus or AWSLambda) and AWS RDS) and TERRAFORM (and excluding the Camunda and Kong – business process implementation is a deliverable for 2nd sprint):

   - Prosumer
   - UtilityOperator
   - AssetLink
   - Telemetry Ingestion
   - Flexibility Emission
   - Grid Balancing
   - Energy Analytics
   - Flexibility Forecasting (AI)

4. Documentation of tests considering all the previous deliverables

5. Documentation for the source code, terraform scripts files, installation procedures, and parametrizations.

### Submission:

One single ZIP file via Fénix containing the report PDF and all the developed code, scripts, and other artifacts.

The deadline for uploading your project in Fénix is strictly defined to **10/5/2026 23h59m**

## 2ND SPRINT

The second sprint reuses the microservices from previous sprint and finalizes the business processes.

### The expected deliverables are:

1. The development of the support business processes using Camunda platform, Kong, and all the required integrations. All the developed microservices should now be used by the business processes:
    - Prosumer Management
    - Utility Operator Management
    - Asset Link Management
    - Telemetry Ingestion
    - Flexibility Emission
    - Grid Balancing Recommendation
    - Energy Analytics
    - Flexibility Forecasting (AI)
2. Documentation of end-to-end tests considering the full business processes execution.
3. Documentation for the source code, terraform script files, BPMN files, installation procedures, and parametrizations.

### Submission:

One single ZIP file via Fénix containing the report PDF and all the developed code, scripts, and other artifacts.

The deadline for uploading your project in Fénix is strictly defined to **7/6/2026 23h59m**